

15-8 安裝 face_recognition（人臉辨識）程式庫與 dlib 工具程式

許多影像辨識、機器學習和數據分析軟體都用到名叫 "dlib" 的工具程式（網址：dlib.net）。dlib 是個已有十多年歷史的開放原始碼專案，主要創作者是 Davis King。dlib 的功能包羅萬象，最早是為了提供跨系統、硬體平台處理多執行緒、聯網、線性代數...等函式的 C++ 語言程式庫。後來陸續加入影像處理、HOG 特徵描述器、深度學習...，並且提供 Python 語言程式介面，因此廣受各領域的程式設計師愛用。另一個知名的開放原始碼人臉辨識程式庫是卡尼基美隆大學的研究生 Brandon Amos 創作的 OpenFace (<https://bit.ly/2eOHlMA>)。

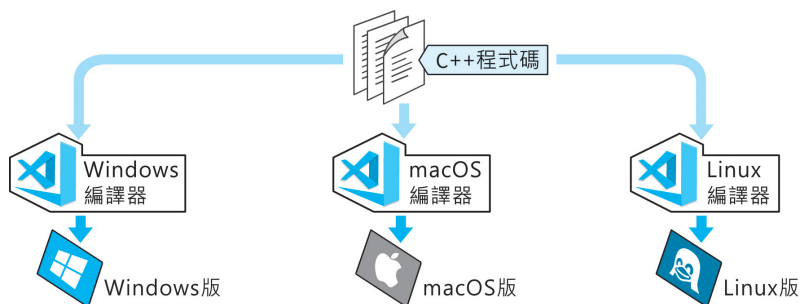
dlib 官網有提供 C++ 和 Python 語言的程式範例，以及 Python API 說明文件。不同於商用的雲端人臉辨識服務，例如 Amazon Rekognition API 和 Microsoft Azure Face API，使用 dlib 進行人臉辨識不需要聯網，所有運算都在本機電腦完成。dlib 包含經過約 300 萬張影像訓練的特徵分析成果，用知名的人像資料集 "Labeled Faces in the Wild"（簡稱 LFW，<http://vis-www.cs.umass.edu/lfw/>）測試，準確率高達 99.38%。

Adam Geitgey 運用 dlib 開發了一個 face_recognition 程式庫，讓人臉辨識程式開發變得輕而易舉，此程式庫的專案網頁 (<https://bit.ly/2oRgoGT>) 也有簡體中文版說明文件。face_recognition 具有三大用途：

- 人臉偵測
- 人臉辨識
- 人臉關鍵點標記 (Face Landmarks)

安裝 face_recognition 程式庫時，它會自動下載、編譯、安裝 dlib。但就像應用軟體和 App 有平台版本之分（例如：Windows 版的遊戲無法安裝在 Android

手機)，那些用 C++ 程式語言編寫的程式庫原始碼（如：dlib），必須要使用對應系統平台的 C++ 編譯器軟體，編譯成可執行檔之後，才能在系統平台上運作。



在 Windows 系統上安裝 dlib 程式庫有兩種方式：

1

下載他人事先編譯好的 .whl 格式，透過 pip 命令直接安裝。優點是簡單方便，缺點是版本稍微舊一點。

2

在電腦上安裝與設置 C++ 編譯環境，自行編譯 dlib 的原始碼再安裝。過程稍微繁瑣一點，但是可以使用最新版。

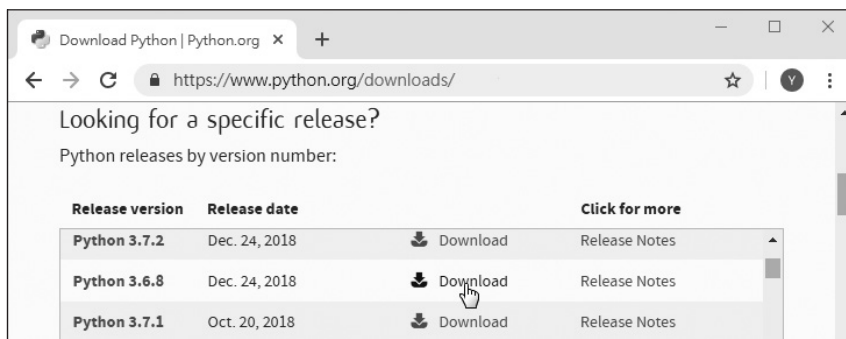
安裝預先編譯好的 dlib 工具程式

dlib 下載頁面 (pypi.org/simple/dlib/) 列舉了原始檔（未編譯）和已編譯好的版本，筆者在撰寫本文時，已編譯的最新版本是 19.8.1 版，而 face_recognition 程式庫所需的 dlib 是 19.7 或更高版本，因此符合需求。

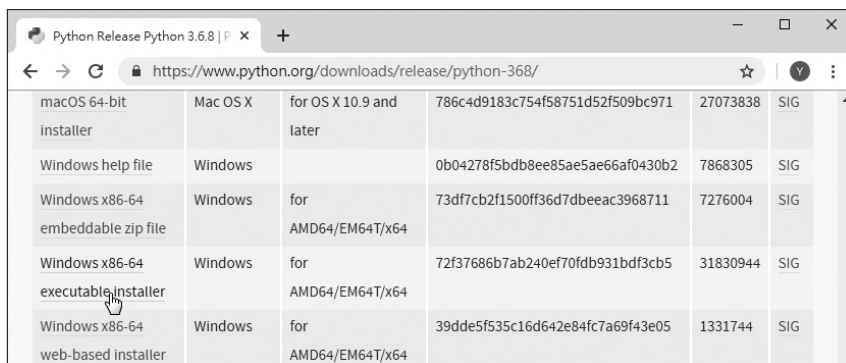


已編譯的版本檔名裡的 `cp36`，代表它要求 Python 執行環境是 3.6.x 版。有些程式庫可以與舊版或新版的 Python 執行環境相容，但 `dlib` 不行。所以安裝這個版本的 `dlib` 之前，你必須先移除電腦上的 Python，再安裝 3.6.x 版。如果讀者發現有新的已編譯 `dlib` 版本，檔名中間有標示 `cp37` 或更新版，那就不用安裝 Python 3.6.x 版了。

Python 官網的下載頁 (www.python.org/downloads/) 列舉了新舊版 Python，請點選 Python 3.6.x 版的 Download (下載) 連結：



接著下載 Python 安裝程式：



Python 3.6.x 版安裝完畢後，假設你下載了 "`dlib-19.8.1-cp36-cp36m-win_amd64.whl`" 並儲存在 D 磁碟的根目錄，請在命令行中，於 D 磁碟路徑執行 `pip` 命令安裝它：

```
D:\> pip install dlib-19.8.1-cp36-cp36m-win_amd64.whl
```

接著安裝 face_recognition 程式庫：

```
pip install face_recognition
```

筆者在兩台電腦，分別以 Python 3.7.x 和 Python 3.6.x 版執行本單元的影像處理和人臉辨識程式，都能執行無誤。

編譯 dlib：下載與安裝 CMake

若要自行在 Windows 平台編譯 dlib，電腦上必須事先安裝兩個軟體：

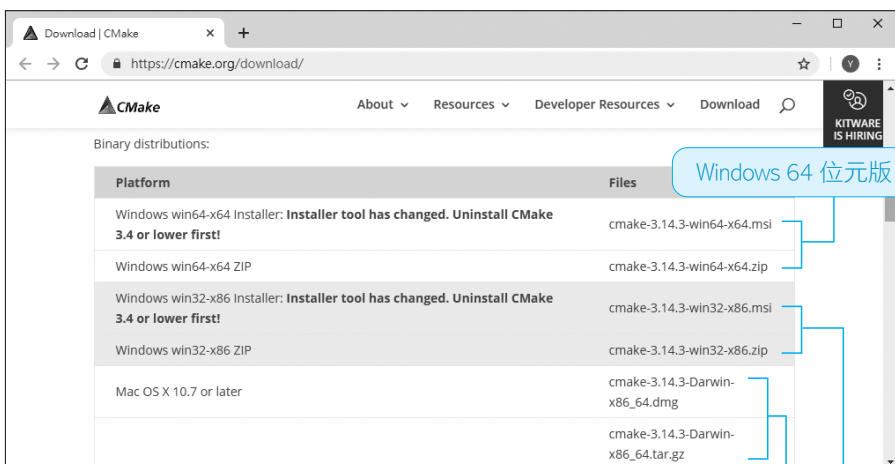
1

CMake 應用程式（免費，有 Windows、macOS 和 Linux 版）

2

微軟的 Visual Studio IDE（整合式開發環境，非第一章安裝的 Code 版）：以下簡稱 VS IDE，筆者安裝的是免費、功能完整的「社群（Community）」版。

macOS 系統只需要安裝 CMake，請參閱下文「在 macOS 系統編譯及安裝 dlib」說明。Windows 使用者請下載 .msi 格式的 CMake 安裝程式 (<https://cmake.org/download/>)。

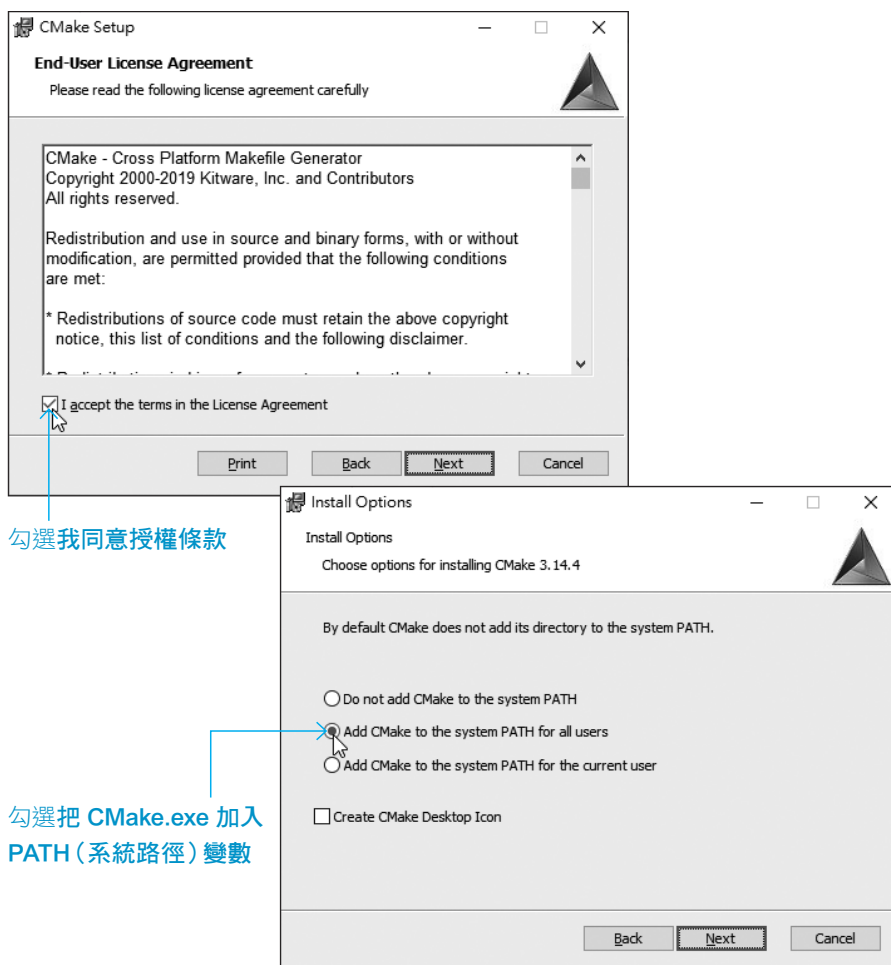


Windows 64 位元版

Mac OS X 版

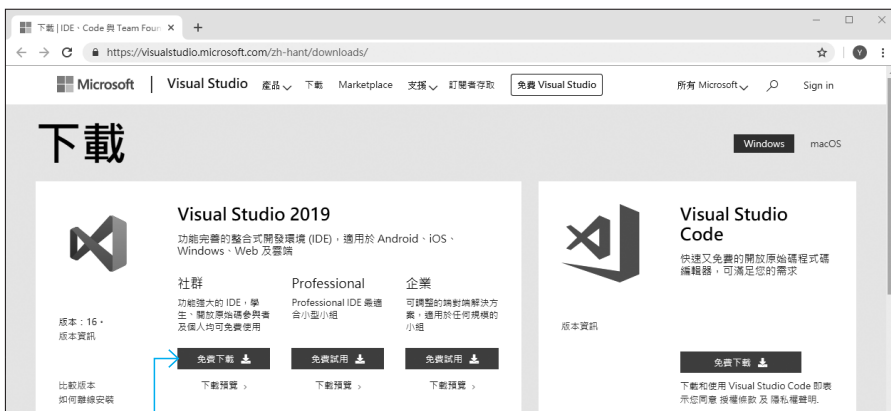
Windows 32 位元版

CMake 的安裝過程大致就是按 **Next (下一步)** 鈕，直到最後 (**Finish**) 完成。一開始必須勾選**我同意授權條款**選項，此外，請務必勾選把 **CMake.exe** 加入 **PATH (系統路徑)** 變數選項，其他程式才能在任何路徑執行它。



在 Windows 系統下載與安裝 VS IDE：編譯 dlib

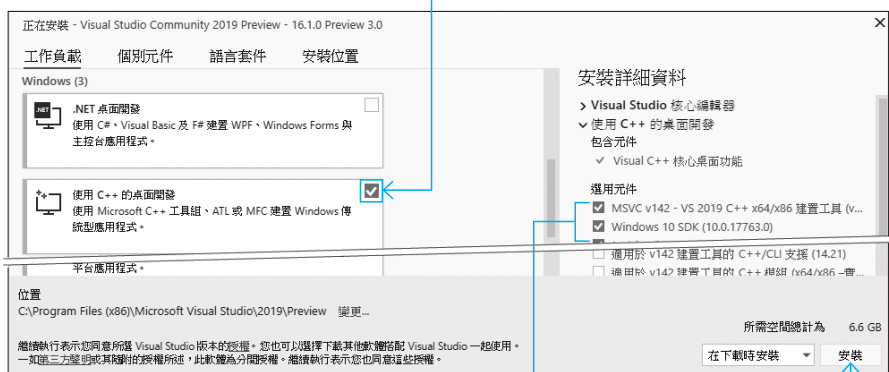
請在 VS IDE 下載頁 (<https://visualstudio.microsoft.com/zh-hant/downloads/>) 選擇社群版：



下載社群版

在 VS IDE 安裝過程中，它會要求你勾選**工作負載**，也就是開發軟體專案的類型，如：網頁程式、雲端 App、桌面應用程式...等，請勾選使用 **C++** 的桌面開發，其他類型的工作負載可選擇性安裝。

1 勾選這一項



2 選用預設勾選的元件即可

3 按下安裝

安裝完畢後，它會自行啟動 VS IDE 軟體，你可以將它關閉。我們只是為了透過它的 C++ 元件來編譯程式，並不需要執行 VS IDE。

在命令提示字元執行 `pip` 命令安裝 `dlib`：

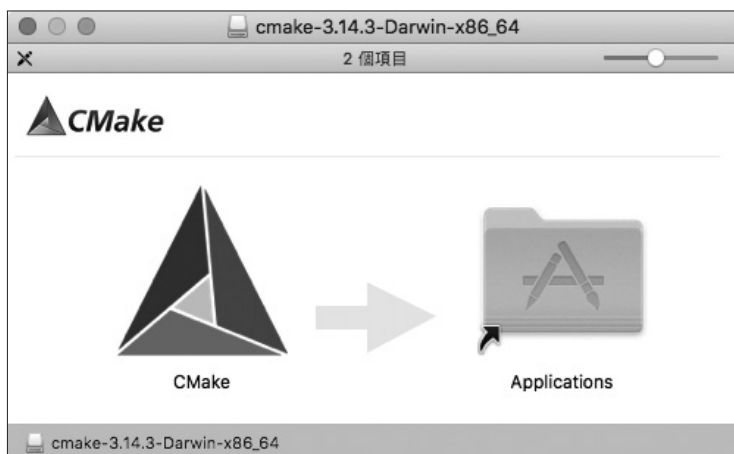
```
pip install dlib
```

或直接執行底下的 `pip` 命令安裝 `face_recognition` 人臉辨識程式庫，在安裝過程中，它會下載 `dlib` 並自行啟動 CMake 和 VS Code 的 C++ 編譯工具程式進行編譯。在筆者的第一代微軟 Surface Pro 電腦 (Intel Core i5-3317U 處理器、4GB 主記憶體)，編譯 `dlib` 約耗時 12 分鐘。

```
pip install face_recognition
```

在 macOS 系統編譯及安裝 `dlib`

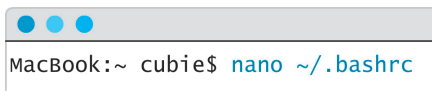
在 macOS 系統編譯 `dlib` 工具程式，電腦上只需要安裝 CMake 軟體並設定它的所在路徑。請參閱上一節說明，下載 .dmg 格式、macOS 版的 CMake 軟體，下載之後雙按它，將開啟如下的視窗畫面：



請把其中的 CMake 拖曳到箭頭右邊的 **Applications (應用程式)** 文件夾，隨即完成安裝。接著開啟 **Terminal (終端機)**，依下列步驟，把 CMake 軟體的所在路徑紀錄在系統的 PATH 路徑：

1

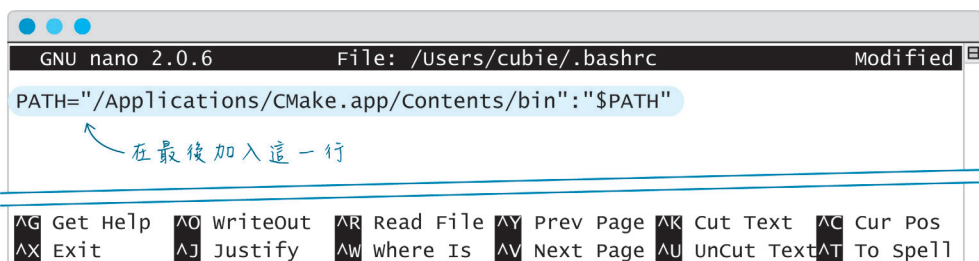
使用 nano 文字編輯器開啟
使用者的執行環境設定檔
(~/.bashrc)：



```
MacBook:~ cubie$ nano ~/.bashrc
```

2

如果此檔案是空白的，請直接在其中輸入底下一行敘述，把 CMake
應用程式所在路徑加入 PATH (路徑) 變數；若檔案不是空白的，請
將它加入最後一行：

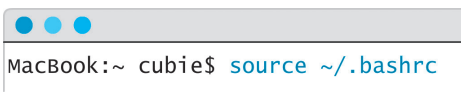


3

按下 **Ctrl** + **O** 鍵寫入 (存檔)，按下 **Enter** 確定，再按下 **Ctrl** +
X 鍵關閉。

4

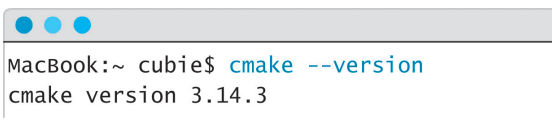
執行 **source** 命令，讓
~/.bashrc 檔的設定內容
即刻生效。



```
MacBook:~ cubie$ source ~/.bashrc
```

5

在任何路徑執行底下的敘述，它將傳回 CMake 應用程式版本，代表
PATH 路徑設定成功。



```
MacBook:~ cubie$ cmake --version
cmake version 3.14.3
```

現在可以執行 pip 命令安裝 face_recognition 程式庫了，在安裝過程，pip 會自
動下載必要的 dlib，然後啟動 CMake 進行編譯：

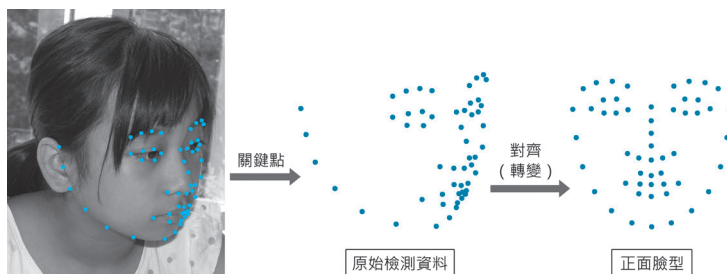
```
pip3 install face_recognition
```

15-9 人臉偵測

使用上文的維奧拉-瓊斯或 HOG 演算法偵測到影像中的人臉之後，為了辨別不同人，電腦需要區別不同面孔的特徵。**人臉關鍵點偵測** (keypoint detection，也稱為 landmark detection) 是一種演算法，可以在影像中的人臉上標示眉毛、眼、口、鼻的位置和臉部輪廓，如下圖左所示。關鍵點越多越精確，但運算需求也越高，在不要求精確，或者像右下圖般在人臉上戴上眼鏡、面具等應用，也可以簡化到 5 個關鍵點。



人臉關鍵點偵測主要用在**臉部對齊** (face alignment)，代表把偵測到的關鍵點，對應成正面臉型的關鍵數據。



「臉部對齊」功能對「人臉辨識」很重要，因為從不同角度觀看同一個人臉，得到的量測數據將不一樣，像嘴唇的長度就差很多，如果直接用影像的測量數據比對同一人的不同角度照片，機器將會判定是不同人。先把偵測到的關鍵點對齊成正面再比對，才能獲得正確的結果。

使用 face_recognition 程式庫標示臉部特徵

本單元程式使用 face_recognition 程式庫的兩個方法，載入影像並找出影像中的人臉特徵部位及其座標：

- **load_image_file**：載入影像檔並轉成 Numpy 陣列格式。
- **face_landmarks**：找出影像中的人臉與特徵部位，包括：chin（下巴）、left_eye（左眼）、top_lip（上唇）...以及它們的座標位置。

接著透過 PIL 程式庫在影像上描繪出特徵點，完整程式碼如下，筆者把各部位的特徵英文和中文名稱用字典格式儲存備用；face_recognition 程式庫的名字有點長，筆者將它簡寫成 fr：

```
import face_recognition as fr ← 載入人臉識別程式庫並簡寫成 fr
from PIL import Image, ImageDraw

feature_name = {
    'chin': '下巴', 'left_eyebrow': '左眉', 'right_eyebrow': '右眉',
    'nose_bridge': '鼻樑', 'nose_tip': '鼻尖', 'left_eye': '左眼',
    'right_eye': '右眼', 'top_lip': '上唇', 'bottom_lip': '下唇'}
```

```
img_path = 'D:\\sophia.jpg' ← 影像檔路徑
img = fr.load_image_file(img_path) ← 載入影像檔並轉成 Numpy 陣列
face_list = fr.face_landmarks(img) # 找出影像中的人臉與特徵點
print("在相片中找到{}張臉".format(len(face_list)))
```

每個臉孔圖像物件的 keys() 方法將傳回特徵部位的名稱，例如 "chin", "left_eye" ...等，所以程式可以用它來查詢對應的中文名字：

```
pil_img = Image.fromarray(img) # 從 Numpy 陣列建立影像
d = ImageDraw.Draw(pil_img) # 準備在影像上描繪圖像

for marks in face_list: ← 從每張臉
    for f in marks.keys(): ← 取出每個特徵
        print("{}的特徵點座標：{}".format(feature_name[f], marks[f]))
        d.line(marks[f], width=5) ← 取得特徵的中文
pil_img.show() ← 根據特徵點座標，繪製5像素粗的直線。
```

此程式將讀取 D 磁碟的 sophia.jpg 圖檔，執行結果如下：



終端機也將顯示各特徵部位的名字與座標：

```

CA 命令提示字元
D:\opencv>python landmark.py
使在相片中找到1張臉
下巴的特徵點座標：[(248, 236), (251, 271), (256, 307), ...]
左肩的特徵點座標：[(276, 203), (295, 182), (323, 175), ...]
:
  
```

用矩形線框標示影像中的人臉

臉孔偵測軟體常見的一項功能，是在找到人臉之處用一個矩形線框標示出來。這項功能主要仰賴 face_recognition 程式庫的 **face_location**（直譯為「臉部位置」）方法達成。執行本節的程式碼之後，右下圖兩張臉孔將被藍色線框包圍：

```

import face_recognition as fr
img = fr.load_image_file("kids.jpg")
face_loc = fr.face_locations(img)
  
```

載入影像檔

取得臉部位置

[(119, 589, 504, 204),
(562, 1061, 883, 740)]
上 左 下 右



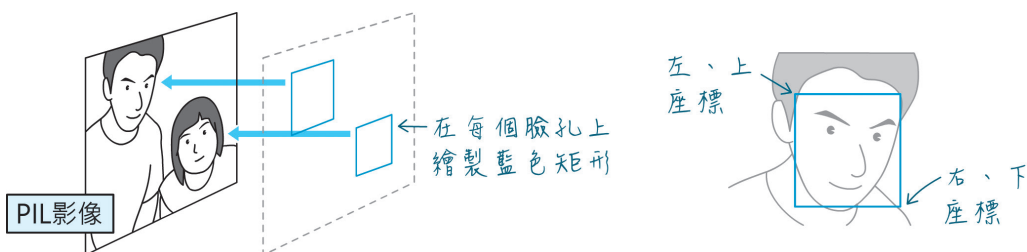
face_locations() 將傳回偵測到的所有臉部座標；如果沒有偵測人臉，則傳回空列表。以右上圖的影像為例，face_loc 列表將包含兩個元素。底下的程式使用 PIL 程式庫顯示影像，並且依據座標值在人像上繪製藍色矩形。

```
import face_recognition as fr
from PIL import Image, ImageDraw ← 包含繪圖指令的模組
:
pil_img = Image.fromarray(img) ← 從Numpy陣列建立影像物件

for loc in face_loc:
    top, right, bottom, left = loc
    draw = ImageDraw.Draw(pil_img) ← 建立「繪製影像」物件
    draw.rectangle([left, top, right, bottom], outline="blue")
    pil_img.show() ← 顯示影像
```

取出座標：上、右、下、左
建立「繪製影像」物件
矩形座標：左、上、右、下
外框線：藍色
等同
outline=(0,0,255)

ImageDraw 物件的 **rectangle()** 方法用於繪製矩形，它的 **outline** 參數代表外框線，可用 HTML 網頁標準的色彩名稱（如:red, yellow, blue, ...），或者用 3 個元素的元組來設定 RGB 色彩值，例如:(0, 0, 255)，代表紅、綠色都關閉、藍色亮度開到最大。

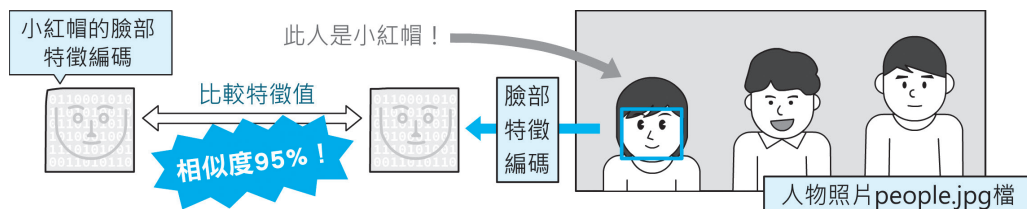


15-10 人臉識別程式

「人臉識別」代表不僅要找出影像中的人臉，還要知道那個人是誰。為了達成這個目標，我們要先準備好辨識目標的正面大頭照，不必是辦理證件用的那種大頭照，可以從一般照片擷取下來，只要五官能清楚辨識；此外，還要標示大頭照的名字，最基本的方法是用人名當作大頭照的檔名。



接著，用程式讀取大頭照並產生臉部特徵編碼，以後的程式將能用這個特徵編碼跟其他影像中的人臉特徵值比對，得知影像裡面是否包含目標人士。



產生人臉識別特徵編碼

本單元將建立一個可以讀取大頭照相片和人名，產生特徵編碼之後，將已知人員資料存成 Pickle 格式檔案的程式。為了方便管理，筆者使用試算表紀錄識別對象的姓名、照片檔名以及 RFID 標籤碼（此欄可先留空，日後再填寫）。RFID 的全名是「無線射頻識別系統」，相當於無線條碼，廣泛運用於悠遊卡、國道 ETC 收費辨識系統、學生卡之類的無線感應卡，每一張無線感應卡都包含唯一識別碼，讀取此識別碼所需的硬體裝置和程式說明，請參閱「附錄 C」的門禁系統實作。

	A	B	C
1	name	大頭目	小紅帽
2	pict	eric.jpg	sophia.jpg
3	RFID	0x22dc36b	0xb0e20055

第一行是資料名稱

姓名

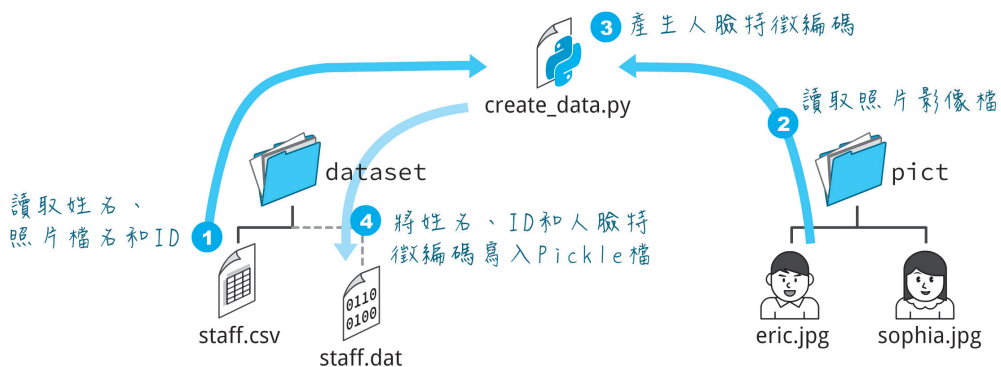
大頭照檔名

轉存

name,大頭目,小紅帽
pict,eric.jpg,sophia.jpg
RFID,0x22dc36b,0xb0e20055

staff.csv檔

把試算表檔案匯出成 staff.csv 檔，存入 dataset 資料夾，大頭照則存入 pict 資料夾。接下來，編寫一個 create_data.py 程式，解析 CSV 檔案裡的資料並讀入大頭照，產生臉部特徵編碼，最後以 Pickle 格式存入 dataset 資料夾，供日後的人臉識別程式使用。



create_data.py 程式碼的主要內容如下，首先宣告儲存使用者資料的空白 staff 字典：

```

staff = {
    'name': [], # 姓名
    'pict': [], # 照片檔名
    'RFID': [],
    'encode': [] # 人臉特徵編碼
}

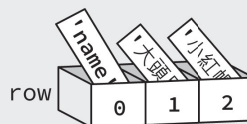
```

底下是讀入 CSV 檔，並依資料名稱，把資料值存入 staff 字典的程式片段：

```
csv_file = 'D:\\dataset\\staff.csv' # CSV檔案路徑
```

```
with open(csv_file, encoding='utf-8') as f:
    csv_data = csv.reader(f, delimiter=',')
```

```
for row in csv_data:
    _key = row[0] # 第0行是資料名稱
    _data = row[1:] # 第1到最後一行是資料
```



```
if _key == 'RFID':
    staff[_key] = [int(i, base=16) for i in _data]
else:
    staff[_key] = _data
```

原始值是16進位
把_data的每個元素轉成整數；
語法說明請參閱附錄A，A-2頁。

底下的 for 迴圈，將逐一讀取紀錄在 staff 的 'pict' 影像檔名，透過 face_recognition 程式庫的 load_image_file() 方法讀入影像檔，再透過 face_encodings() 方法產生臉部編碼，最後把編碼存入 staff 字典的 'encode' 資料。

```
for pic in staff['pict']:
    img = fr.load_image_file(pict_path+pic) # 讀取人像照片檔
    encoding = fr.face_encodings(img)[0] # 產生人臉特徵編碼
    staff['encode'].append(encoding) # 存入 staff 字典的
                                     'encode' 資料

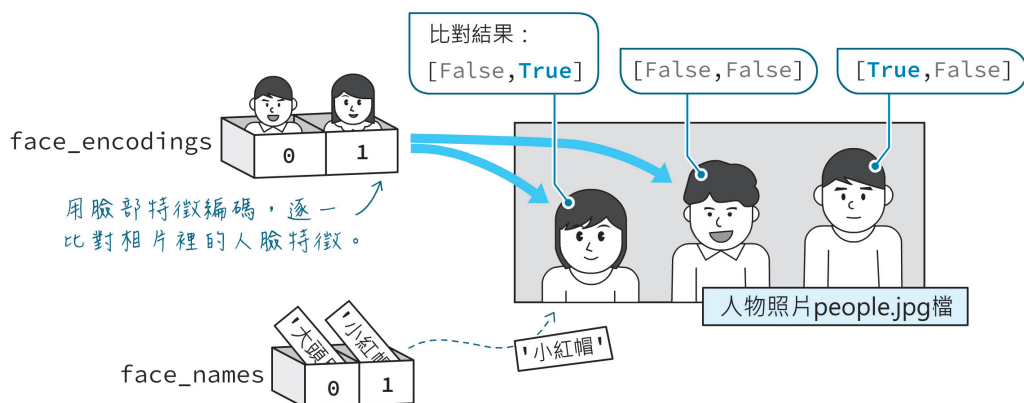
pickle_file = 'D:\\dataset\\staff.dat'
with open(pickle_file, 'wb') as f: # 用 Pickle 儲存 staff 字典資料
    pickle.dump(staff, f)
```

執行 create_data.py 程式，它將在 dataset 資料夾寫入包含人名和臉部編碼的 staff.dat 檔。

編寫人臉識別程式

本單元程式將用藍色矩形框標示影像中的面孔並顯示已知人士的名字，不知名人士則標示成「路人甲」。下圖展示本程式的運作流程，讀取人物照片

(people.jpg) 之後，透過 face_recognition 程式庫找出相片中的臉孔位置，並替每張臉建立特徵編碼，然後跟上一節建立的已知人士的特徵編碼比對，就能標示面孔的名字。



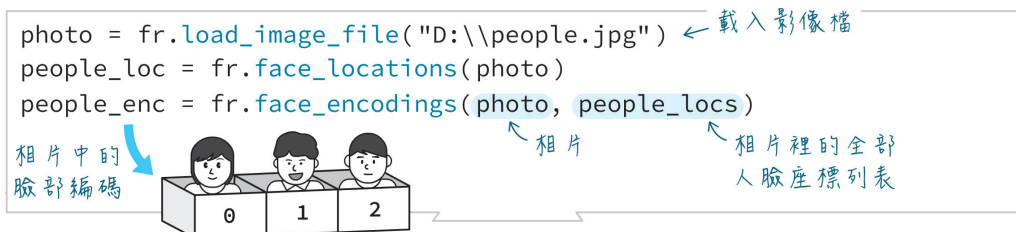
程式首先取出之前建立的特徵編碼資料：

```
import face_recognition as fr
from PIL import Image, ImageDraw, ImageFont
import pickle

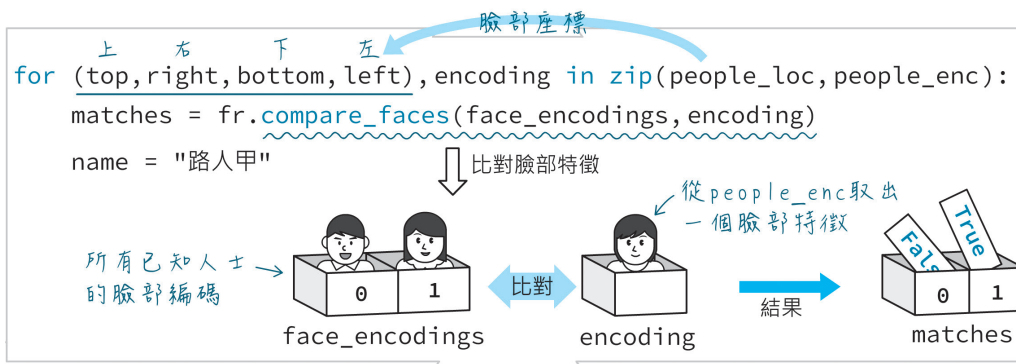
# 包含臉部編碼和人名的 Pickle 檔案
dataset_file = 'D:\\dataset\\staff.dat'
# 包含中文的思源黑體字
font_file = "D:\\font\\NotoSansCJKtc-Regular.otf"
_font = ImageFont.truetype(font_file, 15) # 產生 15 像素大小的字體

with open(dataset_file, 'rb') as f: # 讀取 Pickle 檔
    d = pickle.load(f)
    face_encodings = d['encode'] # 取出臉部特徵編碼列表
    face_names = d['name'] # 取出人名列表
```

接著載入相片 (people.jpg 檔)，並替其中的臉孔建立特徵編碼。範例照片包含三個人像，所以 face_locations() 方法將傳回 3 組臉孔的座標位置列表，face_encoding() 方法則傳回 3 組臉孔的特徵編碼列表。



使用 `for` 迴圈，對相片裡的每個臉孔跟已知臉孔的編碼相比較，結果存入 `matches` 列表，發現相符時，存入 `True`，否則存入 `False`。相片左邊女生的特徵編碼和 `face_encoding` 的第 1 個元素相符，所以 `matches` 的第 1 個元素值為 `True`：



比對臉部特徵的 `compare_faces()` 函式，有個調整容許誤差的 `tolerance`（容錯率）參數，預設值為 0.6。容錯率介於 0~1，數值越低越嚴謹（精確），但所需運算時間也越長。底下是指定容錯率的例子：

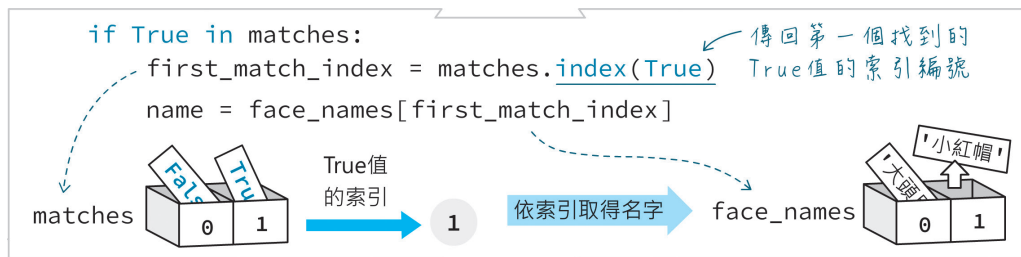
```

mathes = fr.compare_faces(face_encodings, encoding,
    tolerance=0.5)

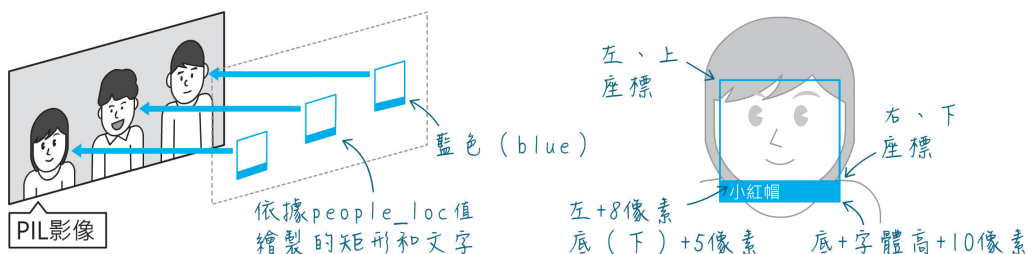
```

實測人臉辨識程式時，同一家族的臉孔使用預設的容錯率，經常會辨識錯誤，調整成 0.5 就不易出錯。

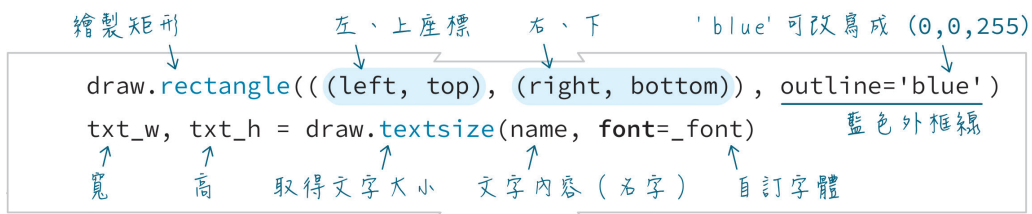
假如 matches 列表包含 True 值，則透過 index() 方法取得它的元素編號，藉此得知此人的名字：



最後在臉孔位置繪製兩個藍色矩形，一個包圍臉部，一個放在臉部底下，填入藍色底和白色文字的人名：



首先繪製包圍臉部的矩形框線，並且估算顯示名字的文字所需的寬和高：



然後繪製藍色矩形並填入白色文字：

```
draw.rectangle(((left, bottom+txt_h+10), (right, bottom)),
               填入藍色 → fill='blue', outline='blue')
draw.text((left+8, bottom+5), name, fill='white', font=_font)
```

文字左邊和上方各留8和5像素的留白

文字填入白色

本單元程式 (face_rec.py 檔) 執行結果如下：

